

CL-2006 Operating System Lab

Code Explanation Guide

Lab 08: Multithreading & Synchronization with Pthreads
Lab 10: Process Synchronization using Semaphores

*Complete Line-by-Line Code Walkthrough
with Theoretical Concepts & Quiz Preparation*

Topics Covered:

Pthreads · Mutex Locks · Race Conditions
Semaphores · Thread Ordering · Counting Semaphores
Shared Memory · Synchronization Primitives

Contents

PART I: Lab 08 – Multithreading & Synchronization with Pthreads	2
1 Key Theoretical Concepts (Lab 08)	2
1.1 What Threads Share vs. What They Don't	2
1.2 The Three Core Pthread Functions	2
2 Example 1: Your First Thread Program (Pthreads_1.c)	4
2.1 Line-by-Line Explanation	4
3 Example 2: Threads Running in Parallel – Infinite Loop (Pthreads_2.c)	7
3.1 Line-by-Line Explanation	7
4 Example 3: Passing an Integer & Getting a Return Value (Pthreads_3.c)	9
4.1 Line-by-Line Explanation	9
5 Example 4: Passing a Struct – Multiple Arguments (Pthreads_4.c)	12
5.1 Line-by-Line Explanation	12
6 Example 5: Passing an Array to a Thread (Pthreads_5.c)	14
6.1 Line-by-Line Explanation	14
7 Example 6: Thread Attributes (pthread_properties.c)	16
7.1 Line-by-Line Explanation	17
8 Example 7: Race Condition in Action – The Bug (mutex.c)	19
8.1 Line-by-Line Explanation	20
9 Example 8: Race Condition Fixed with Mutex (mut_solved.c)	21
9.1 Line-by-Line Explanation	22
PART II: Lab 10 – Process Synchronization using Semaphores	24
10 Key Theoretical Concepts (Lab 10)	24
10.1 The Two Fundamental Operations	24
10.2 Three Types of Semaphores by Initial Value	24
10.3 Named vs Unnamed Semaphores	25
11 Implementation of Semaphores – Step by Step	26
11.1 Step 0: Required Headers and Compilation	26
11.2 Step 1: Declaration	26
11.3 Step 2: Initialization – <code>sem_init()</code>	26
11.4 Step 3: Waiting – <code>sem_wait()</code>	27
11.5 Step 4: Posting – <code>sem_post()</code>	27
11.6 Step 5: Destroying – <code>sem_destroy()</code>	28
11.7 The Complete Pattern in One Place	28
12 Exercise 1: Basic Semaphore and Thread Ordering (Semaphores_1.c)	29
12.1 Line-by-Line Explanation	29

13 Exercise 2: Binary Semaphore as Mutex (Semaphores_2.c)	31
13.1 Line-by-Line Explanation	32
14 Exercise 3: Thread Ordering with Two Semaphores (Semaphores_3.c)	34
14.1 Line-by-Line Explanation	35
15 Exercise 4: Counting Semaphore (Semaphores_4.c)	37
15.1 Line-by-Line Explanation	38
16 Common Mistakes to Avoid – Both Labs	40
17 Quick Reference Summary – Quiz Cheat Sheet	41
17.1 Pthread Functions Quick Reference	41
17.2 Semaphore Functions Quick Reference	41
17.3 Compilation Commands	41

PART I: Lab 08 – Multithreading & Synchronization with Pthreads

Key Theoretical Concepts (Lab 08)

Process vs Thread – The Core Difference

A **process** (created with `fork()`) gets its own *separate* memory space – like opening a brand new restaurant with its own kitchen, staff, and inventory.

A **thread** (created with `pthread_create()`) shares everything with its parent process – like hiring a new chef in the *same* kitchen. Threads share global variables, heap memory, and open files, but each has its own **stack** (local variables) and **program counter**.

What Threads Share vs. What They Don't

Memory Area	Shared?	Example
Global variables	YES	<code>int counter = 0;</code> at top of file
Heap (malloc)	YES	<code>int *p = malloc(...);</code>
Open files	YES	<code>FILE *f = fopen(...);</code>
Code (text segment)	YES	All threads call same functions
Stack (local vars)	NO – separate	<code>int i;</code> inside a thread function
Registers	NO – separate	Program counter, CPU state
Thread ID	NO – separate	<code>pthread_self()</code> gives unique value

Warning

Why this matters for coding: Because global variables and heap are *shared*, two threads can read and write the same variable simultaneously – causing **race conditions**. This is why we need **mutex locks**.

The Three Core Pthread Functions

Function Call	What It Does	Analogy
<code>pthread_create(&tid, attr, func, arg)</code>	Creates a new thread running <code>func(arg)</code>	Hire a new chef
<code>pthread_join(tid, &retval)</code>	Wait for thread to finish	Wait for dish
<code>pthread_exit(retval)</code>	End current thread	Chef says “done”

Compilation Warning – Always Remember!

You MUST add `-lpthread` at the end of your `gcc` command, or you will get: undefined reference to `'pthread_create'`

```
gcc program.c -o program -lpthread    # CORRECT
gcc program.c -o program                # WRONG -- linker
error!
```

Example 1: Your First Thread Program (Pthreads_1.c)

What This Example Teaches

The absolute minimum thread program. We create **two threads**, each running a different function, and the main thread waits for both to finish using `pthread_join()`.

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5
6 void *printp() {
7     printf("printp\n");
8     return NULL;
9 }
10
11 void *print1() {
12     printf("print1\n");
13     return NULL;
14 }
15
16 int main() {
17     pthread_t t1, t2;
18
19     pthread_create(&t1, NULL, printp, NULL);
20     pthread_create(&t2, NULL, print1, NULL);
21
22     pthread_join(t1, NULL);
23     pthread_join(t2, NULL);
24
25     return 0;
26 }
```

Listing 1: Pthreads_1.c – Creating two simple threads

Line-by-Line Explanation

`#include <stdio.h>`

Includes the standard input/output library. Needed for `printf()` to print messages to the terminal.

`#include <pthread.h>`

Includes the POSIX Threads library header. This gives us access to all `pthread_*` functions, the `pthread_t` type, and mutex functions. **Required for any thread program.**

```
#include <stdlib.h>
```

Provides general utility functions like `exit()`, `malloc()`, etc.

```
#include <unistd.h>
```

Provides POSIX system calls like `sleep()` and `usleep()`. Not strictly needed here but included for future use.

```
void *printp()
```

Defines the function that Thread 1 will execute. **All thread functions MUST have the signature** `void *(function_name)(void *arg)`. The return type is `void*` (a generic pointer) because threads can return any data type through a pointer.

```
printf("printp\n");
```

Thread 1 prints its message. Since both threads run “simultaneously” (the OS decides the order), you might see either thread print first.

```
return NULL;
```

The thread finishes and returns `NULL` (no return value). This is like `pthread_exit(NULL)`. Always return something – even `NULL` – from a thread function.

```
pthread_t t1, t2;
```

Declares two **thread ID** variables of type `pthread_t`. Think of `pthread_t` like a process ID (`pid_t`) – it’s a unique identifier for each thread. These are **OUTPUT** variables – `pthread_create` will fill them in.

```
pthread_create(&t1, NULL, printp, NULL);
```

Creates Thread 1. The four arguments:

&t1 – pointer to thread ID (output: gets filled with Thread 1’s ID)

NULL – use default thread attributes (stack size, scheduling, etc.)

printp – the function this thread will run (function pointer)

NULL – argument to pass to `printp()` – `NULL` means no argument

Returns 0 on success, non-zero on failure.

```
pthread_create(&t2, NULL, print1, NULL);
```

Creates Thread 2 running `print1()`. Both threads are now running concurrently with main and with each other.

```
pthread_join(t1, NULL);
```

Makes **main thread wait** until Thread 1 finishes. Without this, main would reach `return 0` immediately and kill both threads before they could print anything.

NULL as second arg means we don’t care about the return value.

```
pthread_join(t2, NULL);
```

Similarly waits for Thread 2 to finish before main exits.

Why Threads Don't Always Run in Order

Even though we created `t1` before `t2`, the OS scheduler might run `t2` first. This is called **non-deterministic execution**. Run this program 5 times and you may see different output orders – this is *normal behavior*, not a bug.

Expected Output

```
printp  
print1  
(Order may vary between runs)
```


Example 2: Threads Running in Parallel – Infinite Loop (Pthreads_2.c)

What This Example Teaches

Proves that threads truly run **simultaneously**. Two threads loop forever printing different messages with `sleep()` delays – you can watch them interleave in real time.

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5
6 void *printp1() {
7     while (1) {
8         printf("T1 = HELLO\n");
9         sleep(2);
10    }
11 }
12
13 void *printp2() {
14     while (1) {
15         sleep(2);
16         printf("T2 = WORLD\n");
17     }
18 }
19
20 int main() {
21     pthread_t t1, t2;
22     pthread_create(&t1, NULL, printp1, NULL);
23     pthread_create(&t2, NULL, printp2, NULL);
24     pthread_join(t1, NULL);    // Never returns (infinite loop)
25     pthread_join(t2, NULL);
26     return 0;
27 }
```

Listing 2: Pthreads_2.c – Parallel execution with infinite loops

Line-by-Line Explanation

```
while (1) {
```

Creates an **infinite loop** – this thread runs forever until the program is killed with Ctrl+C. This is used to demonstrate continuous parallel execution.

```
printf("T1 = HELLO\n");
```

Thread 1 prints first, then sleeps.

```
sleep(2);
```

Pauses Thread 1 for 2 seconds. During this sleep, the OS can run Thread 2. `sleep()` is from `unistd.h` and takes seconds.

```
void *printp2() { while (1) { sleep(2); printf("T2 = WORLD\n"); } }
```

Thread 2 sleeps FIRST (2 seconds), then prints. This offset creates a visible alternating pattern in the output. Both threads sleep 2 seconds but at different times in their loops.

```
pthread_join(t1, NULL); // Never returns
```

Since Thread 1 loops forever, this `pthread_join` call will never return. The program will run until you press Ctrl+C. This is intentional to keep the main thread alive while the other two threads run.

Expected Output (press Ctrl+C to stop)

```
T1 = HELLO
T2 = WORLD
T1 = HELLO
T2 = WORLD
...
```

Key Insight about `sleep()`

When a thread calls `sleep()`, it does **not block other threads**. Thread 1 sleeping does NOT pause Thread 2. The OS simply lets Thread 2 run while Thread 1 is sleeping. This is the beauty of concurrency.

Example 3: Passing an Integer & Getting a Return Value (Pthreads_3.c)

What This Example Teaches

How to pass data **INTO** a thread and how to get data **OUT** of a thread. Uses `void*` casting to pass integers and retrieve return values.

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5
6 void *printp1(void *parm) {
7     printf("HELLO = %d\n", *(int *)parm);
8     return (void *)2;
9 }
10
11 void *printp2(void *parm) {
12     printf("Some Value = %d\n", *(int *)parm);
13     return (void *)13;
14 }
15
16 int main() {
17     pthread_t t1, t2;
18     int val1 = 1, val2 = 2;
19
20     pthread_create(&t1, NULL, printp1, (void *)&val1);
21     pthread_create(&t2, NULL, printp2, (void *)&val2);
22
23     void *rc, *get;
24     pthread_join(t1, &rc);
25     pthread_join(t2, &get);
26
27     printf("Thread 1 returned: %d\n", (int)(long)rc);
28     printf("Thread 2 returned: %d\n", (int)(long)get);
29     return 0;
30 }
```

Listing 3: Pthreads_3.c – Passing integers and getting return values

Line-by-Line Explanation

`void *printp1(void *parm)`

The thread function now takes `void *parm` – a generic pointer that can hold *any type* of data. This is how you pass arguments to threads: through this single `void*` parameter.

```
printf("HELLO = %d\n", *(int *)parm);
```

This line **casts and dereferences** the void pointer:

Step 1: (int *)parm – cast void* to int* (tell compiler “this pointer points to an int”)

Step 2: *(int *)parm – dereference to get the actual integer value
This gives us the integer that was passed in.

```
return (void *)2;
```

Returns the integer 2 as a void*. We cast the integer directly to a pointer (not recommended for large data but works for small integers). The caller retrieves it with pthread_join.

```
int val1 = 1, val2 = 2;
```

We store our values in **separate local variables**. Each thread gets a pointer to its own variable. **IMPORTANT:** Never pass the same variable address to multiple threads – they’d all modify the same memory!

```
pthread_create(&t1, NULL, printp1, (void *)&val1);
```

Passes the **address** of val1 to Thread 1. We cast &val1 (which is int*) to void* because pthread_create expects void*. Inside the thread, it gets cast back to int*.

```
void *rc, *get;
```

Declare two void* variables to **collect return values** from threads. These will be filled by pthread_join.

```
pthread_join(t1, &rc);
```

Waits for Thread 1 to finish AND stores Thread 1’s return value into rc. We pass &rc (address of the pointer) because pthread_join needs to write to our variable.

```
printf("Thread 1 returned:  %d\n", (int)(long)rc);
```

The cast (int)(long)rc:

First: cast void* to long (avoids truncation on 64-bit systems where pointers are 8 bytes)

Then: cast long to int to get the integer value
This retrieves the 2 that Thread 1 returned.

Expected Output

```
HELLO = 1
Some Value = 2
Thread 1 returned: 2
Thread 2 returned: 13
```

The void* Pattern – Master This!

Passing into thread:

```
int x = 5;  
pthread_create(&t, NULL, f,  
              (void*)&x);
```

Receiving in thread:

```
int val = *(int *)arg;
```

Always: pass the **address**, cast to **void***; inside thread: cast back to original type, then dereference.

Example 4: Passing a Struct – Multiple Arguments (Pthreads_4.c)

What This Example Teaches

The **most common real-world pattern**: when you need to pass multiple values of different types to a thread, bundle them in a **struct** and pass a pointer to the struct.

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5
6 typedef struct {
7     int    int_arg;
8     float  float_arg;
9     char   str_arg[100];
10 } ThreadArgs;
11
12 void* loop(void* arg) {
13     ThreadArgs* args = (ThreadArgs*)arg;
14     printf("int_arg = %d, float_arg = %.2f, str_arg = %s\n",
15           args->int_arg, args->float_arg, args->str_arg);
16     return NULL;
17 }
18
19 int main() {
20     pthread_t t1;
21     ThreadArgs args;
22     args.int_arg = 42;
23     args.float_arg = 3.14;
24     strcpy(args.str_arg, "Hello, world!");
25
26     pthread_create(&t1, NULL, loop, &args);
27     pthread_join(t1, NULL);
28     return 0;
29 }
```

Listing 4: Pthreads_4.c – Passing a struct to a thread

Line-by-Line Explanation

```
typedef struct { int int_arg; float float_arg; char str_arg[100]; }
ThreadArgs;
```

Defines a custom struct to bundle all arguments together. Since `pthread_create` only accepts ONE `void*` argument, we use a struct to package multiple values. `typedef` lets us write `ThreadArgs` instead of `struct ThreadArgs`.

```
ThreadArgs* args = (ThreadArgs*)arg;
```

Inside the thread function: cast the received `void*` pointer back to `ThreadArgs*`. Now we have a typed pointer to our struct and can access its fields.

```
printf("...", args->int_arg, args->float_arg, args->str_arg);
```

Use **arrow operator** (`->`) because `args` is a *pointer* to a struct. This is equivalent to `(*args).int_arg`. Prints all three fields.

```
args.int_arg = 42; args.float_arg = 3.14; strcpy(args.str_arg, "Hello, world!");
```

Populates the struct fields in main before creating the thread. `strcpy` is used for the char array (cannot assign strings with `=` in C).

```
pthread_create(&t1, NULL, loop, &args);
```

Passes the **address of the struct** (`&args`) as the thread argument. No cast to `void*` needed here (C automatically converts pointer types in function calls).

Lifetime Warning

The struct `args` is declared in `main()` on the stack. The thread receives a pointer to it. If `main()` returns before the thread finishes, the struct is destroyed and the thread accesses invalid memory. Always `pthread_join` before returning from main!

Expected Output

```
int_arg = 42, float_arg = 3.14, str_arg = Hello, world!
```

Example 5: Passing an Array to a Thread (Pthreads_5.c)

What This Example Teaches

Arrays in C are *already pointers*, so passing them to threads is straightforward. The thread receives an `int*` and walks through the array.

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 void *array(void *arg) {
5     int *arr = (int *)arg;
6     for (int i = 0; i < 5; i++) {
7         printf("%d ", *arr);
8         arr++;
9     }
10    printf("\n");
11    return NULL;
12 }
13
14 int main() {
15     int arr2[5] = {1, 2, 3, 4, 5};
16     pthread_t t1;
17     pthread_create(&t1, NULL, array, (void *)arr2);
18     pthread_join(t1, NULL);
19     return 0;
20 }
```

Listing 5: Pthreads_5.c – Passing an array to a thread

Line-by-Line Explanation

```
int *arr = (int *)arg;
```

Cast `void*` back to `int*`. Now `arr` points to the first element of the array (`arr2[0]` in `main`).

```
printf("%d ", *arr);
```

Dereference `arr` to get the current element's value. On first iteration: `*arr = arr2[0] = 1`.

```
arr++;
```

Advance the pointer to the next integer in memory (moves forward by `sizeof(int)` = 4 bytes). Next iteration: `*arr = arr2[1] = 2`, and so on.

```
pthread_create(&t1, NULL, array, (void *)arr2);
```

Pass the array name `arr2` directly. In C, an array name decays to a pointer to its first element (`arr2` is same as `&arr2[0]`). Cast to `void*` for `pthread_create`.

Expected Output

1 2 3 4 5

Array vs Struct for Passing Data

Use a **plain array pointer** when all data is the same type (homogeneous). Use a **struct** when you have mixed types or need to pass additional metadata (like the array size, start index, end index).

Example 6: Thread Attributes (pthread_properties.c)

What This Example Teaches

How to customize thread behaviour using `pthread_attr_t`. Attributes control the thread's stack size, scheduling policy, and whether it can be joined (joinable) or runs independently (detached).

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4
5 void* myThreadFunc(void* arg) {
6     for (int i = 0; i < 5; i++) {
7         printf("Thread %d running\n", (int)(long)arg);
8         usleep(100000);
9     }
10    return NULL;
11 }
12
13 int main() {
14     pthread_t thread1, thread2;
15     pthread_attr_t attr;
16
17     // Step 1: Initialize attributes with defaults
18     pthread_attr_init(&attr);
19
20     // Step 2: Set the attributes
21     pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);
22     pthread_attr_setstacksize(&attr, 2 * 1024 * 1024); // 2 MB
23
24     // Step 3: Create threads with custom attributes
25     pthread_create(&thread1, &attr, myThreadFunc, (void*)1);
26     pthread_create(&thread2, &attr, myThreadFunc, (void*)2);
27
28     // Step 4: Join threads
29     pthread_join(thread1, NULL);
30     pthread_join(thread2, NULL);
31
32     // Step 5: Clean up attributes
33     pthread_attr_destroy(&attr);
34     printf("Main thread finished.\n");
35     return 0;
36 }
```

Listing 6: pthread_properties.c – Using thread attributes

Line-by-Line Explanation

```
pthread_attr_t attr;
```

Declares a thread **attributes object**. This is a struct that holds all configurable thread settings. Before use, it must be initialized with `pthread_attr_init`.

```
pthread_attr_init(&attr);
```

Initializes the attributes object with **default values**. Always call this first before setting any individual attributes. Failure to initialize first causes undefined behavior.

```
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);
```

Sets whether threads created with this attribute can be joined. `PTHREAD_CREATE_JOINABLE` (default) means we CAN call `pthread_join()` on this thread. The alternative `PTHREAD_CREATE_DETACHED` means the thread cleans up itself automatically when done (but you can't join it).

```
pthread_attr_setstacksize(&attr, 2 * 1024 * 1024);
```

Sets the thread's stack to 2 MB. Each thread gets its own stack for local variables and function calls. If you have deeply recursive functions or large local arrays, increase this. $2 * 1024 * 1024 = 2,097,152 \text{ bytes} = 2 \text{ MB}$.

```
pthread_create(&thread1, &attr, myThreadFunc, (void*)1);
```

Creates Thread 1 with our **custom attributes** instead of the default NULL. Note we pass `&attr` (address of attr object) not NULL.

```
pthread_attr_destroy(&attr);
```

Releases resources held by the attributes object. Call this **after** creating all threads (the created threads are not affected). Forgetting this leaks memory.

```
usleep(100000);
```

Sleeps for 100,000 microseconds = 0.1 seconds. `usleep()` (from `unistd.h`) takes microseconds. Used to slow down threads so output is readable.

Attribute	What It Controls	Default	When to Change
Detach state	Can you join this thread?	JOINABLE	For fire-and-forget threads
Stack size	Local var memory	2–8 MB	Deep recursion
Scheduling policy	How OS schedules it	SCHED_OTHER	Real-time systems
Priority	Runs more/less often	0	High-priority tasks

Tip

In most student programs, passing `NULL` to `pthread_create` (default attributes) is perfectly fine. Custom attributes are only needed for specialized use cases.

Example 7: Race Condition in Action – The Bug (mutex.c)

What This Example Teaches

Demonstrates a **race condition** – the bug that happens when multiple threads modify shared data without protection. This is intentionally broken code to illustrate the problem.

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5
6 int counter = 0;    // SHARED global -- all threads read and
   write this!
7
8 void *printp1(void *parm) {
9     int i;
10    for (i = 0; i < 5; i++) {
11        counter++;    // DANGER: multiple threads doing this at
   the same time!
12        printf("thread %d counter %d\n", *(int*)parm, counter);
13    }
14    return NULL;
15 }
16
17 int main() {
18     int a = 1, b = 2, c = 3;
19     pthread_t t1, t2, t3;
20
21     pthread_create(&t1, 0, printp1, (void*)&a);
22     pthread_create(&t2, 0, printp1, (void*)&b);
23     pthread_create(&t3, 0, printp1, (void*)&c);
24
25     pthread_join(t1, 0);
26     pthread_join(t2, 0);
27     pthread_join(t3, 0);
28
29     printf("Final counter: %d (expected 15)\n", counter);
30     return 0;
31 }
```

Listing 7: mutex.c – Race condition (BUGGY CODE – intentionally broken)

Line-by-Line Explanation

```
int counter = 0;
```

A **global variable** – shared by all threads. This is the dangerous variable. Because it's global (outside any function), it lives in the shared data segment, accessible by every thread.

```
for (i = 0; i < 5; i++) { counter++; ... }
```

Each of the 3 threads increments `counter` 5 times. Expected total: $3 \times 5 = 15$. But because threads run concurrently, they can interfere with each other.

```
counter++;
```

This looks like one operation but at the CPU level it is **THREE steps**:

Step 1: READ `counter` from memory into a CPU register

Step 2: ADD 1 to the register value

Step 3: WRITE the new value back to memory

If Thread A does Step 1 (reads 5) and then Thread B does Step 1 (also reads 5) before Thread A does Step 3, both will write 6 – and one increment is **lost**!

```
pthread_create(&t1, 0, printp1, (void*)&a);
```

Note: second argument is 0 instead of NULL. These are equivalent in C – 0 converts to a null pointer. Both mean “use default attributes”.

The Race Condition

Expected final counter: **15**. Actual result varies – could be 13, 14, or sometimes 15 (by luck). Each run may give a different answer. This unpredictability is the hallmark of a race condition. The program is non-deterministic.

Why `counter++` is NOT Atomic

Thread A reads `counter = 5`

Thread B reads `counter = 5` <-- both read SAME old value!

Thread A adds 1, writes 6

Thread B adds 1, writes 6 <-- writes 6 again, not 7!

Result: `counter = 6` instead of 7. One increment is LOST.

Example 8: Race Condition Fixed with Mutex (mut_solved.c)

What This Example Teaches

How to fix the race condition using a **mutex lock**. Only one thread can hold the lock at a time – others must wait. This ensures the counter is always correct.

```

1  #include <stdio.h>
2  #include <pthread.h>
3  #include <stdlib.h>
4  #include <unistd.h>
5
6  int counter = 0;           // Shared variable
7  pthread_mutex_t mutex;    // THE LOCK
8
9  void *printp1(void *parm) {
10     int i;
11     pthread_mutex_lock(&mutex);    // LOCK: "I'm going in,
12                                     // everyone wait!"
13
14     for (i = 0; i < 5; i++) {
15         counter++;                // Now SAFE -- only one
16                                     // thread at a time
17         printf("thread %d counter %d\n", *(int*)parm, counter);
18         usleep(1000);
19     }
20
21     pthread_mutex_unlock(&mutex);  // UNLOCK: "I'm done, next
22                                     // person can go!"
23     return NULL;
24 }
25
26 int main() {
27     int a = 1, b = 2, c = 3;
28     pthread_t t1, t2, t3;
29
30     pthread_mutex_init(&mutex, NULL); // Create the lock
31
32     pthread_create(&t1, 0, printp1, (void*)&a);
33     pthread_create(&t2, 0, printp1, (void*)&b);
34     pthread_create(&t3, 0, printp1, (void*)&c);
35
36     pthread_join(t1, 0);
37     pthread_join(t2, 0);
38     pthread_join(t3, 0);
39
40     pthread_mutex_destroy(&mutex);    // Clean up the lock
41     printf("Final counter: %d\n", counter); // Always 15!
42     return 0;
43 }

```

Listing 8: mut_solved.c – Race condition FIXED with mutex

Line-by-Line Explanation

```
pthread_mutex_t mutex;
```

Declares a **mutex lock** variable. `pthread_mutex_t` is a data type (like `int` or `pthread_t`) that represents a lock. Declared globally so all threads can access it. Think of it as a bathroom lock: only one person inside at a time.

```
pthread_mutex_init(&mutex, NULL);
```

Initializes (creates) the mutex. Must be called BEFORE any thread tries to use it. Second argument is mutex attributes (NULL = default). This is analogous to installing the lock on a bathroom door.

```
pthread_mutex_lock(&mutex);
```

The thread tries to **acquire** (lock) the mutex:

If mutex is FREE: the calling thread locks it and continues immediately.

If mutex is LOCKED (another thread holds it): the calling thread **blocks** (goes to sleep) and waits until the mutex is released.

Only one thread can hold the lock at a time – this is **mutual exclusion**.

```
for (i = 0; i < 5; i++) { counter++; ... }
```

This code is now the **critical section** – only one thread can execute it at a time because the mutex is locked. No race condition possible here. Thread 1 runs all 5 iterations, then unlocks, then Thread 2 runs all 5, etc.

```
usleep(1000);
```

Sleep 1000 microseconds (1 millisecond) to simulate work. Even though this thread sleeps here, OTHER threads cannot enter the critical section because the mutex is still locked.

```
pthread_mutex_unlock(&mutex);
```

The thread **releases** the lock. If any other threads were blocked waiting in `pthread_mutex_lock`, one of them is now woken up and gets to acquire the lock.

```
pthread_mutex_destroy(&mutex);
```

Frees resources associated with the mutex. Call this **after** all threads have finished (after all `pthread_join` calls). Analogous to removing the lock from the bathroom door.

Function	Analogy	Effect
<code>pthread_mutex_init(&m, NULL)</code>	Install lock	Creates the lock
<code>pthread_mutex_lock(&m)</code>	Enter & lock door	Block if busy, else lock
<code>pthread_mutex_unlock(&m)</code>	Exit & unlock	Wake next waiting thread
<code>pthread_mutex_destroy(&m)</code>	Remove lock	Free kernel resources

Expected Output – Always Correct Now

```
thread 1 counter 1
thread 1 counter 2 ...through 5
thread 2 counter 6 ...through 10
thread 3 counter 11 ...through 15
Final counter: 15
```

Fine-Grained vs Coarse-Grained Locking

In this example, the *entire loop* is inside the lock (coarse-grained). This means threads run one at a time – no real parallelism. In production code, you'd lock *only* the `counter++` line (fine-grained locking) so threads can do their other work in parallel and only wait for the shared variable access.

Golden Rule of Mutex Usage

- If any global variable is **modified** by more than one thread $\Rightarrow$ **protect it with a mutex**
- Read-only shared data (constants) does NOT need a mutex
- Every lock must have a matching unlock – forgetting unlock causes **deadlock**

PART II: Lab 10 – Process Synchronization using Semaphores

Key Theoretical Concepts (Lab 10)

What is a Semaphore?

A **semaphore** is a non-negative integer counter that supports exactly **two atomic operations**: **wait** (decrement) and **post** (increment). Linux guarantees these operations are atomic – they cannot be interrupted halfway, eliminating the race condition problem that `counter++` had.

Think of it like a **parking lot counter**:

Initial value = **3** $\Rightarrow$ 3 parking spaces available.

Each car entering does `sem_wait()` – decrements the counter (1 space taken).

Each car leaving does `sem_post()` – increments the counter (1 space freed).

When counter = 0, the next car to arrive blocks (waits in line) until a space opens.

The Two Fundamental Operations

Operation	What It Does	Think of it as
<code>sem_wait()</code>	Decrement by 1. If already 0, block until someone posts.	“Try to enter – wait if full”
<code>sem_post()</code>	Increment by 1. Wake up a blocked thread if any.	“I’m leaving – next one can enter”

Three Types of Semaphores by Initial Value

Type	Init Value	Behaviour	Use Case
Binary	1	Only 1 thread at a time	Protecting shared variables (like mutex)
Counting	N (>1)	Up to N threads at once	Limited resource pools (e.g., 3 DB connections)
Signal/Order	0	Thread blocks until signalled	Thread ordering / chaining

Named vs Unnamed Semaphores

Named Semaphores (`sem_open()`): Exist as files in the filesystem (`/dev/shm`). Can be shared between *completely separate programs*. Persist until `sem_unlink()` is called.

Unnamed Semaphores (`sem_init()`): Live in memory (in a variable you declare). Used within *one process* to synchronize threads. Destroyed when program exits or `sem_destroy()` is called.

This lab uses unnamed semaphores exclusively.

Implementation of Semaphores – Step by Step

The Complete Semaphore Lifecycle

Every semaphore program follows the same 6 steps. Master these and you can write any semaphore program.

Step 0: Required Headers and Compilation

```
#include <semaphore.h>
```

Provides the `sem_t` data type and all semaphore functions (`sem_init`, `sem_wait`, `sem_post`, `sem_destroy`, `sem_getvalue`). **Required for semaphore programs.**

```
#include <pthread.h>
```

Required because we use POSIX threads alongside semaphores.

```
gcc -o output_name source_file.c -pthread
```

Compilation Flag

Use `-pthread` (NOT `-lpthread`) for semaphore programs. This flag links both the pthread library AND the semaphore library. Without it: **undefined reference** to '`sem_init`' errors.

Step 1: Declaration

```
sem_t semaphore;
```

Declares a semaphore variable. `sem_t` is a data type (like `int`) defined in `semaphore.h`. Declare it as a **global variable** so all thread functions can access it.

```
sem_t sem1, sem2;
```

You can declare multiple semaphores. Each controls a different resource or ordering constraint.

Step 2: Initialization – `sem_init()`

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

```
sem_init(&semaphore, 0, 1);
```

The three parameters:

&semaphore – pointer to your `sem_t` variable (the semaphore to initialize)

0 – pshared: set to 0 for thread sharing within the same process. Non-zero for sharing across separate processes (requires the semaphore to be in shared memory).

1 – the initial value: starting count of the semaphore.

This creates a **binary semaphore** (mutex-like): only 1 thread can enter the critical section.

```
sem_init(&semaphore, 0, 3);
```

Initial value 3: creates a **counting semaphore**. Up to 3 threads can be inside the critical section simultaneously. When all 3 slots are taken, the 4th thread blocks.

```
sem_init(&semaphore, 0, 0);
```

Initial value 0: creates a **signal/ordering semaphore**. The first thread to call `sem_wait()` on this will immediately block (because 0 means no permits available). It will only proceed when another thread calls `sem_post()`.

Initialization Quick Reference

init = 1 $\Rightarrow$ Mutual exclusion (one thread at a time). Use like a mutex.

init = N $\Rightarrow$ Allow N threads simultaneously. Pool of N resources.

init = 0 $\Rightarrow$ Thread ordering. First waiter blocks until someone posts.

Step 3: Waiting – `sem_wait()`

```
int sem_wait(sem_t *sem);
```

```
sem_wait(&semaphore);
```

Atomically decrements the semaphore value by 1.

If value > 0: decrement and return immediately (thread continues).

If value = 0: block the calling thread (put it to sleep). It waits until another thread calls `sem_post()`, which increments the value and wakes one waiting thread.

Atomic: the check-and-decrement happens as ONE indivisible operation – no race condition possible.

Step 4: Posting – `sem_post()`

```
int sem_post(sem_t *sem);
```

```
sem_post(&semaphore);
```

Atomically increments the semaphore value by 1.

If threads were blocked waiting: one of them is woken up and allowed to proceed (it gets the increment).

If no threads waiting: the value simply increases, meaning the next `sem_wait()` won't block.

The Golden Rule

Every `sem_wait()` must be matched with exactly one `sem_post()`.

Missing a `sem_post()` (e.g., due to an early `return`) leaves waiting threads blocked forever. This is a **deadlock** – the program hangs indefinitely.

Step 5: Destroying – sem_destroy()

```
int sem_destroy(sem_t *sem);
```

```
sem_destroy(&semaphore);
```

Releases kernel resources associated with the semaphore. **Call only after all threads using the semaphore have finished** (after all `pthread_join` calls). Not calling this leaks resources. Do NOT destroy a semaphore while threads are still waiting on it.

The Complete Pattern in One Place

```
1 #include <semaphore.h>
2 #include <pthread.h>
3
4 sem_t sem;                                // Step 1: Declare (global)
5
6 int main() {
7     sem_init(&sem, 0, 1);                  // Step 2: Initialize
8     // ... create threads ...
9     // In thread function:
10    sem_wait(&sem);                        // Step 3: Wait (lock)
11    // ... critical section ...
12    sem_post(&sem);                        // Step 4: Post (unlock)
13    // Back in main, after pthread_join:
14    sem_destroy(&sem);                     // Step 5: Destroy
15 }
```

Listing 9: Complete semaphore lifecycle template

Exercise 1: Basic Semaphore and Thread Ordering (Semaphores_1.c)

What This Exercise Teaches

The simplest use of a semaphore: **controlling execution order**. Thread 1 is forced to run *after* Thread 2 by initializing the semaphore to 0 – Thread 1 blocks immediately and only proceeds when Thread 2 signals it.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4 #include <semaphore.h>
5
6 sem_t sem;           // Global semaphore
7 int counter = 0;     // Shared variable
8
9 void* thread1() {
10     sem_wait(&sem);   // Blocks here until sem > 0
11     counter++;
12     printf("TH1=%d\n", counter);
13     sem_post(&sem);   // Signal done
14 }
15
16 void* thread2() {
17     sem_post(&sem);    // Unblocks Thread 1
18     counter++;
19     printf("TH2=%d\n", counter);
20     sem_post(&sem);
21 }
22
23 int main() {
24     sem_init(&sem, 0, 0); // Initial value = 0 (Thread 1 will
                           // block)
25     pthread_t t1, t2;
26     pthread_create(&t1, NULL, thread1, NULL);
27     pthread_create(&t2, NULL, thread2, NULL);
28     pthread_join(t1, NULL);
29     pthread_join(t2, NULL);
30     return 0;
31 }

```

Listing 10: Semaphores_1.c – Basic semaphore and thread ordering

Line-by-Line Explanation

```
sem_t sem;
```

Global semaphore variable, shared by both threads.

```
int counter = 0;
```

Shared integer – both threads modify it. Note: in this exercise, the counter is NOT fully protected (a race condition still exists here – fixed in Exercise 2).

```
sem_init(&sem, 0, 0);
```

Initializes semaphore with value **0**. This means “no permits available”. The very first thread to call `sem_wait()` will block immediately.

```
sem_wait(&sem); // Blocks here until sem > 0
```

Thread 1 calls `sem_wait()`. Since `sem = 0`, Thread 1 **immediately blocks** and goes to sleep. It can only proceed when another thread calls `sem_post()` on this same semaphore.

```
counter++; printf("TH1=%d\n", counter);
```

Thread 1 increments and prints. This only happens **AFTER** Thread 2 has called `sem_post()`.

```
sem_post(&sem); // Unblocks Thread 1
```

Thread 2's first action is to post the semaphore. This increments `sem` from 0 to 1, which **wakes Thread 1** from its blocked state. Thread 2 then proceeds with its own work.

```
counter++; printf("TH2=%d\n", counter);
```

Thread 2 does its work. But notice: Thread 2 increments `counter` **WITHOUT** holding the semaphore – this is the unfixed race condition in this exercise.

Expected Output

```
TH2=1    (Thread 2 runs first)
```

```
TH1=2    (Thread 1 runs after Thread 2 posts)
```

(Exact counter values may vary due to the unfixed race condition)

How Thread Ordering Works with Semaphore = 0

1. Main creates both threads. Both start running.
2. Thread 1 calls `sem_wait(&sem)` – `sem=0`, so Thread 1 **sleeps**.
3. Thread 2 calls `sem_post(&sem)` – `sem` goes from 0 to 1, Thread 1 **wakes up**.
4. Thread 1 runs (after Thread 2 has already started).
5. Result: Thread 2 always runs before Thread 1's main work.

Exercise 2: Binary Semaphore as Mutex (Semaphores_2.c)

What This Exercise Teaches

A binary semaphore (init=1) is **functionally identical to a mutex**. `sem_wait()` acts as lock and `sem_post()` acts as unlock. Guarantees the final counter is always correct.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4 #include <semaphore.h>
5
6 #define NUM_THREADS 5
7 #define NUM_INCREMENTS 20
8
9 int counter = 0;
10 sem_t semaphore;    // Binary semaphore (mutex)
11
12 void* increment_counter(void* arg) {
13     sem_wait(&semaphore);           // Lock -- only one
14                                     thread proceeds
15     for (int i = 0; i < NUM_INCREMENTS; ++i) {
16         int temp = counter;
17         temp = temp + 1;
18         counter = temp;
19         printf("Thread %ld -> counter = %d\n", (long)arg,
20               counter);
21     }
22     sem_post(&semaphore);           // Unlock -- next thread
23                                     proceeds
24     return NULL;
25 }
26
27 int main() {
28     pthread_t threads[NUM_THREADS];
29     sem_init(&semaphore, 0, 1);      // Initial value 1 =
30                                     binary semaphore
31
32     for (long i = 0; i < NUM_THREADS; i++)
33         pthread_create(&threads[i], NULL, increment_counter, (
34             void*)i);
35
36     for (int i = 0; i < NUM_THREADS; i++)
37         pthread_join(threads[i], NULL);
38
39     sem_destroy(&semaphore);
40     printf("Final counter: %d\n", counter);
41     return 0;
42 }

```

Listing 11: Semaphores_2.c – Binary semaphore as mutex

Line-by-Line Explanation

```
#define NUM_THREADS 5
```

A preprocessor constant. `#define` tells the compiler to replace every occurrence of `NUM_THREADS` with 5 before compiling. Using named constants is better than magic numbers – change 5 here to change the whole program.

```
#define NUM_INCREMENTS 20
```

Each thread will increment the counter 20 times. Total expected = $5 \times 20 = 100$.

```
sem_init(&semaphore, 0, 1);
```

Initial value 1: binary semaphore. Only 1 thread can be inside the critical section at a time. The first thread to call `sem_wait()` gets in immediately (decrements 1 to 0). All others block.

```
sem_wait(&semaphore); // Lock
```

First thread: sem goes from 1 to 0, proceeds. Second thread: sem is 0, **blocks**. Third, fourth, fifth threads: all block in a queue. They will be released one by one as each finishes.

```
int temp = counter; temp = temp + 1; counter = temp;
```

This is the explicit three-step increment (showing what `counter++` does internally: read, add 1, write back). Inside the lock, this is safe – no race condition.

```
sem_post(&semaphore); // Unlock
```

The current thread finishes all 20 increments and releases the lock. The next blocked thread is woken up and gets to run its 20 increments.

```
pthread_t threads[NUM_THREADS];
```

Array of thread IDs. Using an array lets us create and join threads in a loop.

```
for (long i = 0; i < NUM_THREADS; i++) pthread_create(...);
```

Creates all 5 threads in a loop. Each thread receives its index `i` as an argument (cast to `void*`). All 5 start running but only one can hold the semaphore.

```
for (int i = 0; i < NUM_THREADS; i++) pthread_join(...);
```

Waits for all 5 threads to finish before destroying the semaphore and printing the final count.

Expected Output (always exactly 100)

```
Thread 0 -> counter = 1
Thread 0 -> counter = 2    ... (20 lines for Thread 0)
Thread 1 -> counter = 21   ... (20 lines for Thread 1)
...
Final counter: 100
```

Semaphore vs Mutex – Key Difference

A **mutex** can only be unlocked by the *same thread* that locked it.

A **semaphore** can be posted by *any* thread – even a different one from who called `sem_wait()`.

This makes semaphores more flexible for signalling patterns (as seen in Exercises 1 and 3).

Exercise 3: Thread Ordering with Two Semaphores (Semaphores)

What This Exercise Teaches

Uses **two semaphores** to enforce a strict, deterministic execution order: Thread 2 ALWAYS runs before Thread 1, regardless of OS scheduling. This is the classic **producer-consumer** pattern in its simplest form.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <semaphore.h>
4 #include <pthread.h>
5
6 sem_t sem1, sem2;    // Two semaphores for ordering
7 int counter = 0;
8
9 void* thread1() {
10     sem_wait(&sem1);    // Blocked until Thread 2 posts sem1
11     counter++;
12     printf("TH1=%d\n", counter);
13     sem_post(&sem1);
14     return NULL;
15 }
16
17 void* thread2() {
18     sem_wait(&sem2);    // sem2=1, so proceeds immediately
19     counter++;
20     printf("TH2=%d\n", counter);
21     sem_post(&sem2);    // Release sem2
22     sem_post(&sem1);    // Signal Thread 1 to proceed
23     return NULL;
24 }
25
26 int main() {
27     sem_init(&sem1, 0, 0);    // Thread 1 blocked initially
28     sem_init(&sem2, 0, 1);    // Thread 2 starts immediately
29     pthread_t t1, t2;
30     pthread_create(&t1, NULL, thread1, NULL);
31     pthread_create(&t2, NULL, thread2, NULL);
32     pthread_join(t1, NULL);
33     pthread_join(t2, NULL);
34     sem_destroy(&sem1);
35     sem_destroy(&sem2);
36     return 0;
37 }

```

Listing 12: Semaphores_3.c – Guaranteed thread ordering with two semaphores

Line-by-Line Explanation

```
sem_t sem1, sem2;
```

Two separate semaphore variables. **sem1** controls Thread 1's entry, **sem2** controls Thread 2's entry.

```
sem_init(&sem1, 0, 0);
```

sem1 starts at **0**: Thread 1 will block immediately when it calls `sem_wait(&sem1)`.

```
sem_init(&sem2, 0, 1);
```

sem2 starts at **1**: Thread 2 can proceed immediately when it calls `sem_wait(&sem2)` (decrements from 1 to 0 and continues).

```
sem_wait(&sem1); // Thread 1 - Blocked until Thread 2 posts sem1
```

Thread 1 tries to enter but `sem1=0`, so it **immediately blocks**. It will sleep here until Thread 2 calls `sem_post(&sem1)`.

```
sem_wait(&sem2); // Thread 2 - sem2=1, proceeds immediately
```

Thread 2 calls `sem_wait(&sem2)`. Since `sem2=1` (not 0), Thread 2 decrements it to 0 and **proceeds without blocking**.

```
sem_post(&sem2); // Release sem2
```

Thread 2 re-increments `sem2` back to 1. This restores the semaphore for potential future use (good practice).

```
sem_post(&sem1); // Signal Thread 1 to proceed
```

Thread 2's most important line: increments `sem1` from 0 to 1, which **wakes Thread 1** from its blocked state. Thread 1 can now acquire `sem1` and continue.

```
sem_destroy(&sem1); sem_destroy(&sem2);
```

Clean up both semaphores after all threads finish.

Execution Flow Trace

1. `sem1=0, sem2=1`. Both threads created.
2. Thread 1: `sem_wait(&sem1) → sem1=0`, **Thread 1 blocks**.
3. Thread 2: `sem_wait(&sem2) → sem2 goes 1→0`, Thread 2 **continues**.
4. Thread 2: `counter++; printf("TH2=1");` **Thread 2 prints first**.
5. Thread 2: `sem_post(&sem2) → sem2 goes 0→1`.
6. Thread 2: `sem_post(&sem1) → sem1 goes 0→1`, **wakes Thread 1**.
7. Thread 1: proceeds from `sem_wait(&sem1)`, `sem1 goes 1→0`.
8. Thread 1: `counter++; printf("TH1=2");` **Thread 1 prints second**.

Expected Output – ALWAYS in this order

TH2=1

TH1=2

This is deterministic – unlike Example 1 from Lab 08, this ordering is guaranteed every single run.

Exercise 4: Counting Semaphore (Semaphores_4.c)

What This Exercise Teaches

A **counting semaphore** allows *more than one thread* into the critical section simultaneously – up to the initial count *N*. Perfect for modeling **limited resource pools** (e.g., 3 database connections, 3 printers, 3 download slots).

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <semaphore.h>
4 #include <unistd.h>
5
6 #define NUM_THREADS 5
7
8 sem_t semaphore;    // Counting semaphore
9
10 void* access_resource(void* arg) {
11     int thread_id = (int)(long)arg;
12     sem_wait(&semaphore);    // Decrement count; block if count
                               // == 0
13     // Up to 3 threads can be here at the same time
14     printf("Thread %d is accessing the resource\n", thread_id);
15     sleep(1);
16     printf("Thread %d is leaving the resource\n", thread_id);
17     sem_post(&semaphore);    // Increment count; wake a waiting
                               // thread
18     return NULL;
19 }
20
21 int main() {
22     pthread_t threads[NUM_THREADS];
23     sem_init(&semaphore, 0, 3);    // Allow up to 3 threads
                                     // simultaneously
24
25     for (long i = 0; i < NUM_THREADS; i++)
26         pthread_create(&threads[i], NULL, access_resource, (void
27             *)i);
28
29     for (int i = 0; i < NUM_THREADS; i++)
30         pthread_join(threads[i], NULL);
31
32     sem_destroy(&semaphore);
33     return 0;
34 }
```

Listing 13: Semaphores_4.c – Counting semaphore

Line-by-Line Explanation

```
sem_init(&semaphore, 0, 3);
```

Initial value **3**: up to 3 threads can be inside the critical section (the `sleep(1)` region) at the same time. This models a resource pool with 3 slots.

```
int thread_id = (int)(long)arg;
```

Retrieves the thread's ID from the `void*` argument. Cast chain: `void* → long → int`. The intermediate `long` cast prevents truncation on 64-bit systems.

```
sem_wait(&semaphore); // Decrement count; block if count == 0
```

Each thread tries to enter:

Thread 0: sem goes 3→2. Proceeds.

Thread 1: sem goes 2→1. Proceeds.

Thread 2: sem goes 1→0. Proceeds.

Thread 3: sem=0, **blocks**. Waits.

Thread 4: sem=0, **blocks**. Waits.

```
printf("Thread %d is accessing the resource\n", thread_id);
```

Printed as soon as the thread enters the critical section (after successfully decrementing).

```
sleep(1);
```

Simulates using the resource for 1 second. During this time, up to 3 threads can be sleeping here simultaneously.

```
printf("Thread %d is leaving the resource\n", thread_id);
```

Printed just before releasing the semaphore.

```
sem_post(&semaphore); // Increment count; wake a waiting thread
```

Thread releases its slot. sem increments from 0 to 1. One of the blocked threads (Thread 3 or 4) is woken up and can now enter the critical section.

Expected Output Pattern

```
Thread 0 is accessing the resource
Thread 1 is accessing the resource
Thread 2 is accessing the resource
(Thread 3 and 4 wait here -- all 3 slots taken)
Thread 0 is leaving the resource
Thread 3 is accessing the resource  <-- slot freed, Thread 3 enters
...
```


Counting Semaphore as a Generalization

- `sem_init(&sem, 0, 1)` $\Rightarrow$ Binary semaphore – 1 thread at a time (like mutex)
- `sem_init(&sem, 0, 3)` $\Rightarrow$ 3 threads simultaneously
- `sem_init(&sem, 0, 5)` $\Rightarrow$ All 5 threads run at once (no restriction)
- `sem_init(&sem, 0, 0)` $\Rightarrow$ Signal semaphore – first waiter always blocks

A binary semaphore is just a counting semaphore with max value 1.

Common Mistakes to Avoid – Both Labs

Lab 08 – Most Common Mistakes

1. **Forgetting -lpthread:** `gcc prog.c -o prog` will give undefined reference to `pthread_create`. Fix: `gcc prog.c -o prog -lpthread`
2. **Forgetting `pthread_join()`:** Without it, `main()` exits immediately and kills all threads before they finish. Always join every thread you create.
3. **Passing local loop variable address:**

```
for (int i = 0; i < 5; i++)  
    pthread_create(&t[i], NULL, func, &i); // BUG: all  
    threads share &i!
```

Fix: Use a separate array: `int args[5]; args[i]=i; ... &args[i]`

4. **Not protecting shared data:** Any global variable written by multiple threads needs a mutex. Missing mutex $\Rightarrow$ race condition $\Rightarrow$ wrong results.
5. **Locking for too long:** If you lock the entire function, threads can't run in parallel. Lock only the *minimum* code that accesses shared data (fine-grained locking).

Lab 10 – Most Common Mistakes

1. **Forgetting -pthread flag:** `gcc myfile.c -o myfile` $\Rightarrow$ linker errors. Use `gcc myfile.c -o myfile -pthread`
2. **Missing `sem_post()` after `sem_wait()`:** If a thread exits the function (via early return or `pthread_exit`) without calling `sem_post()`, all other waiting threads are stuck forever = **deadlock**.
3. **Using semaphore before `sem_init()`:** Calling `sem_wait()` on an uninitialized `sem_t` = undefined behaviour (crash or random results). Always `sem_init()` in `main()` before creating threads.
4. **Forgetting `sem_destroy()`:** Not destroying semaphores leaks kernel resources. Always destroy after all threads have joined.
5. **Modifying shared data outside the critical section:** Even one read or write of shared data outside `sem_wait/sem_post` can cause race conditions. The entire read-modify-write sequence must be inside the protected section.

Quick Reference Summary – Quiz Cheat Sheet

Pthread Functions Quick Reference

Function	Purpose / Key Points
<code>pthread_create(&tid, attr, func, arg)</code>	Create thread. Pass NULL for attr (defaults). <code>arg</code> is <code>void*</code> .
<code>pthread_join(tid, &retval)</code>	Wait for thread. NULL for <code>retval</code> if not needed.
<code>pthread_exit(retval)</code>	End current thread with return value.
<code>pthread_self()</code>	Get current thread's ID.
<code>pthread_attr_init(&attr)</code>	Initialize attributes object.
<code>pthread_attr_destroy(&attr)</code>	Free attributes object.
<code>pthread_mutex_init(&m, NULL)</code>	Create mutex lock.
<code>pthread_mutex_lock(&m)</code>	Lock (block if busy).
<code>pthread_mutex_unlock(&m)</code>	Unlock (wake next waiter).
<code>pthread_mutex_destroy(&m)</code>	Free mutex resources.

Semaphore Functions Quick Reference

Function	Purpose / Key Points
<code>sem_init(&sem, 0, value)</code>	Create semaphore. 0=thread-shared. value=initial count.
<code>sem_wait(&sem)</code>	Decrement. Block if value=0. (“lock” / “acquire”)
<code>sem_post(&sem)</code>	Increment. Wake a waiter. (“unlock” / “release”)
<code>sem_destroy(&sem)</code>	Free semaphore after all threads done.
<code>sem_getvalue(&sem, &val)</code>	Read current semaphore value into <code>val</code> .

Compilation Commands

```
# Lab 08 (Pthreads + Mutex):
gcc program.c -o program -lpthread

# Lab 10 (Semaphores):
gcc program.c -o program -pthread

# With warnings (always recommended):
gcc -Wall program.c -o program -lpthread
```

Listing 14: Compilation commands for Lab 08 and Lab 10

Key Conceptual Comparisons for Quiz

	Mutex	Semaphore
Header	pthread.h	semaphore.h
Type	pthread_mutex_t	sem_t
Init	pthread_mutex_init	sem_init
Lock/Wait	pthread_mutex_lock	sem_wait
Unlock/Post	pthread_mutex_unlock	sem_post
Cleanup	pthread_mutex_destroy	sem_destroy
Who unlocks	Only the thread that locked	ANY thread can post
Max users	Always 1	1 (binary) or N (counting)

Three Semaphore Patterns – Know These Cold

1. **Mutual Exclusion (init=1):** `sem_init(&sem, 0, 1)` – only 1 thread in critical section
2. **Thread Ordering (init=0):** `sem_init(&sem, 0, 0)` – Thread A blocks until Thread B signals
3. **Resource Pool (init=N):** `sem_init(&sem, 0, 3)` – up to 3 threads simultaneously

*Good luck on your quiz! Remember: when in doubt, follow the lifecycle –
Declare → Initialize → Wait → [critical section] → Post → Destroy.*